Computation for Analytics

DIANE WOODBRIDGE, PH.D

Contents

Code Conventions

- PEP8
- PEP20

VS Code - AutoPEP8 and Linter

Contents

Code Conventions

- PEP8
- PEP20

VS Code - AutoPEP8 and Linter

Code Conventions

For collaborative environment, using a consistent style provides better code management, readability, understandability and maintenance.

Follow commonly used or internally agreed code conventions.

- Code Conventions : Commonly used and recommended choices as a more readable option.
- PEP (Python Enhancement Proposals)
  - PEP 8 : Style Guide for Python Code
  - PEP 20 : The Zen of Python

Contents

Code Conventions

- **PEP8**
- PEP20

VS Code - AutoPEP8 and Linter

PEP 8 - Style Guide for Python Code

PEP 8 covers code layout, whitespace, naming conventions and other style related topics.

- Code layout
- Whitespaces
- Comments
- Naming Conventions

https://www.python.org/dev/peps/pep-0008

PEP 8 - Style Guide for Python Code

PEP 8 Covers code layout, whitespace, naming conventions and other style related topics.

- **Code layout**
  - Max line length: 79 characters.
  - Indentation
    - Use a tab (4 spaces) per indentation level.
    - Continued lines - long lines should be broken over multiple lines.
  - Blank lines
    - Surround top-level function and class definitions with two blank lines.
    - Method definitions inside a class are surrounded by a single blank line.
  - Importing libraries/packages
    - Imports are always put at the top of the file.
    - Imports should usually be on separate lines.
    - Imports should be grouped in the following order and you should place a line between each group.
      - standard library imports
      - related third party imports
      - local application/library specific imports

https://www.python.org/dev/peps/pep-0008

PEP 8 - Style Guide for Python Code

PEP 8 Covers code layout, whitespace, naming conventions and other style related topics.

- **White spaces**
  - No whitespace
    - Immediately inside parentheses, brackets or braces.
    - Between a trailing comma and a following close parenthesis.
    - Immediately before a comma, semicolon, or colon.
      - However, in a slice the colon acts like a binary operator, and should have equal amounts on either side
    - Immediately before the open parenthesis that starts the argument list of a function call.
    - Immediately before the open parenthesis that starts an indexing.
  - Surround the following binary operators with a single space on either side.
    - assignment (=), augmented assignment (+=, -= etc.), comparisons (==, <, >, !=, <>, <=, >=, in, not in, is, is not), Booleans (and, or, not)
  - If operators with different priorities are used, consider adding whitespace around the operators with the lowest priority(ies).
    - Ex. x = x*2 – 1
  - Don't use spaces around the = sign when it is used to assign a keyword argument or a default parameter value.
  - Multiple statements on the same line are discouraged.

https://www.python.org/dev/peps/pep-0008

PEP 8 - Style Guide for Python Code

PEP 8 Covers code layout, whitespace, naming conventions and other style related topics.

- **Comments**
  - Comments should be complete sentences written in English.
  - Block comments: Block comments generally consist of one or more paragraphs built out of complete sentences, with each sentence ending in a period.
    - Each line of a block comment starts with a # and a single space.
    - block comments are indented to the same level as that code.
  - Inline comments: An inline comment is a comment on the same line as a statement.
    - Inline comments should be separated by at least two spaces from the statement.
    - They should start with a # and a single space.
    - Use inline comments sparingly.
  - Docstring :
    - Write docstrings for all public modules, functions, classes, and methods.
    - For one line docstrings, keep the closing """ on the same line.
    - The """ that ends a multiline docstring should be on a line by itself.

https://www.python.org/dev/peps/pep-0008

Docstring

Docstring

- Describe the operation of the function or class.
  - All modules should normally have docstrings, and all functions and classes exported by a module should also have docstrings.
- Use triple double quotes (" " ") around docstrings.
- Docstrings will be shown in an interactive Python session when you type help(module.function).

```
>>> import lib
>>> help(lib)
```

PEP 8 - Style Guide for Python Code

PEP 8 Covers code layout, whitespace, naming conventions and other style related topics.

- **Naming Conventions**
  - Naming styles:
    - b (single lowercase letter) – Never use 'l' (lowercase letter el), 'O' (uppercase letter oh), or 'I' (uppercase letter eye).
    - B (single uppercase letter)
    - lowercase, lower_case_with_underscores
    - UPPERCASE, UPPER_CASE_WITH_UNDERSCORES
    - CapitalizedWords (or CapWords, or CamelCase)
    - mixedCase
    - _single_leading_underscore: weak "internal use" indicator. (won’t be imported.)

https://www.python.org/dev/peps/pep-0008

PEP 8 - Style Guide for Python Code

PEP 8 Covers code layout, whitespace, naming conventions and other style related topics.

- **Naming Conventions**
  - Packages and modules
    - Packages: Packages should have short, all-lowercase names, although the use of underscores is discouraged.
    - Modules: Modules should have short, all-lowercase names. Underscores can be used in the name.
  - Function, Class and variables
    - Function names: should be lowercase, with words separated by underscores.
    - Variable names: follow the same convention as function names.
    - Class names: should use the CapWords convention.

https://www.python.org/dev/peps/pep-0008

Example

Update ex_pep_example.py to follow PEP8 conventions.

Contents

Code Conventions

- PEP8
- **PEP20**

VS Code - AutoPEP8 and Linter

PEP 20 – Principle for Python's Design

The Zen of Python

Beautiful is better than ugly.

Explicit is better than implicit.

Simple is better than complex.

Complex is better than complicated.

Flat is better than nested.

Sparse is better than dense.

Readability counts.

Special cases aren't special enough to break the rules.

Although practicality beats purity.

Errors should never pass silently.

Unless explicitly silenced.

In the face of ambiguity, refuse the temptation to guess.

There should be one-- and preferably only one --obvious way to do it.

Although that way may not be obvious at first unless you're Dutch.

Now is better than never.

Although never is often better than *right* now.

If the implementation is hard to explain, it's a bad idea.

If the implementation is easy to explain, it may be a good idea.

Namespaces are one honking great idea -- let's do more of those!

https://www.python.org/dev/peps/pep-0020/

Contents

Code Conventions

- PEP8
- PEP20

VS Code - AutoPEP8 and Linter

Too many to remember?

PEP 8 is explicit enough and can be checked programmatically.

- .py

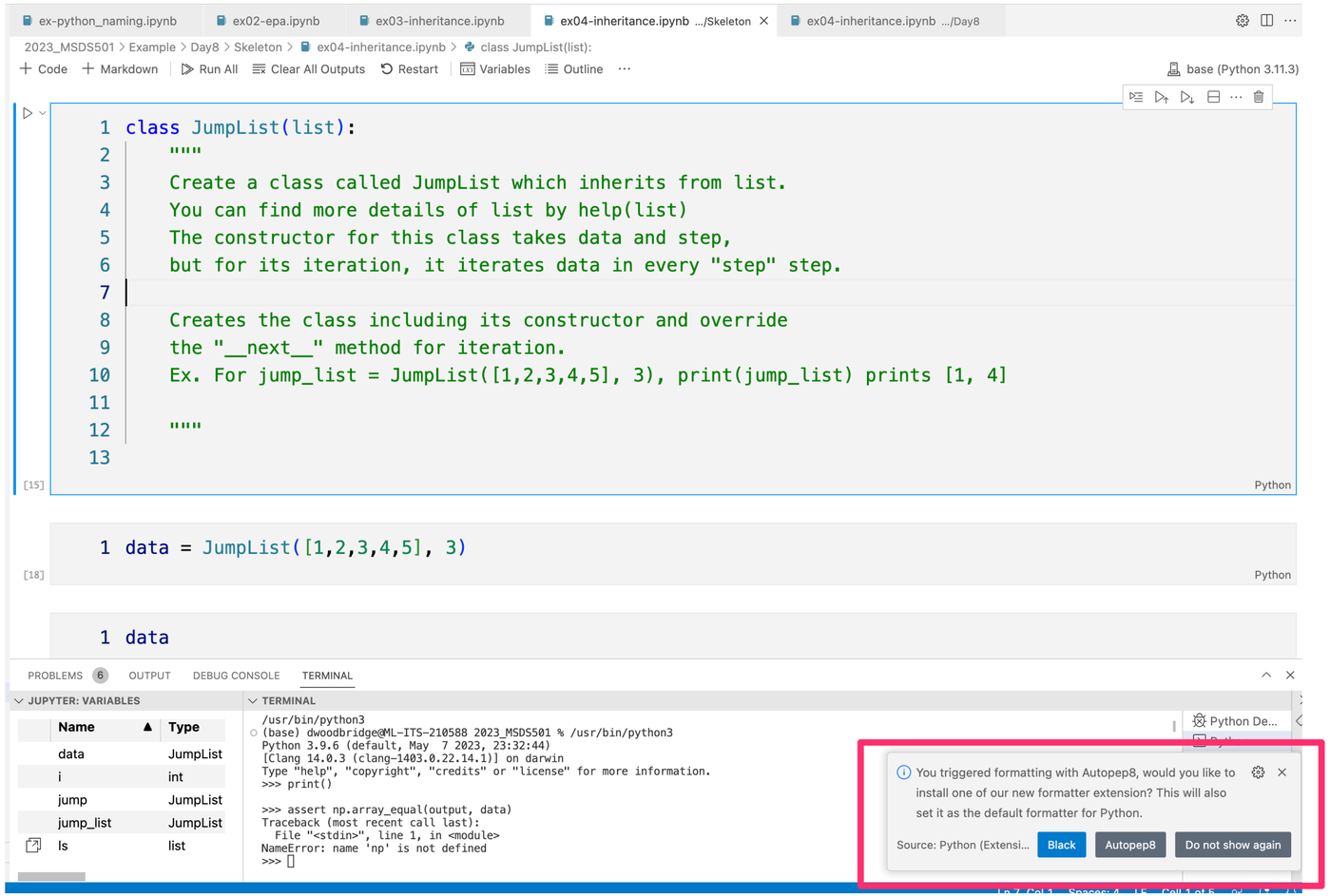
```
$ conda install -c conda-forge pycodestyle
$ pycodestyle your_code.py
```

```
ML-ITS-603436:python_bestpractice dwoodbridge$ pycodestyle ex1.py
ex1.py:27:15: E261 at least two spaces before inline comment
ex1.py:27:16: E262 inline comment should start with '# '
ex1.py:27:80: E501 line too long (96 > 79 characters)
ex1.py:30:1: E265 block comment should start with '# '
ex1.py:31:80: E501 line too long (89 > 79 characters)
```

Auto PEP8 on VSCode

Formatting .ipynb and .py

- Option+Shift+F
  - Then choose AutoPEP8 to install formatter extension
  - If you have take-home coding or ML interview, make sure to at least format your code with PEP8 formatter to provide a visually clean code!



Linter on VSCode

Linter

- Linter highlights syntactic and stylistic mistakes in code and VSCode provides linters for .py files.
  - It does not automatically formats your code but highlights the mistake that you made.
  - While AutoPEP8 only highlights PEP8 issues, linters like Pylint covers other PEP including PEP479, PEP597, PEP3101, etc. The linter will provide you feedback to have better habits on writing a program.
- Installation - Via VS Code Market Place
  - Pylint
  - flake8
  - mypy

https://code.visualstudio.com/docs/python/linting

Contents

Code Conventions

- PEP8
- PEP20

VS Code - AutoPEP8 and Linter

References

PEP8 : https://peps.python.org/pep-0008/

PEP20 : https://peps.python.org/pep-0020/

VS Code Linting - https://code.visualstudio.com/docs/python/linting

Appendix

AUTO PEP8 ON JUPYTER NOTEBOOK

Codestyle Check (autopep8) on Jupyter Notebook

1. **Install *jupyter_contrib_nbextensions*** – A collection of extensions that add functionality to the Jupyter notebook.

   ```
   conda install -c conda-forge jupyter_contrib_nbextensions
   ```

2. **Copy the nbextensions' javascript and css files into the jupyter server's search directory, and edits jupyter config files. '**--user' to install into the user's home jupyter directories.

   ```
   jupyter contrib nbextension install --user
   ```

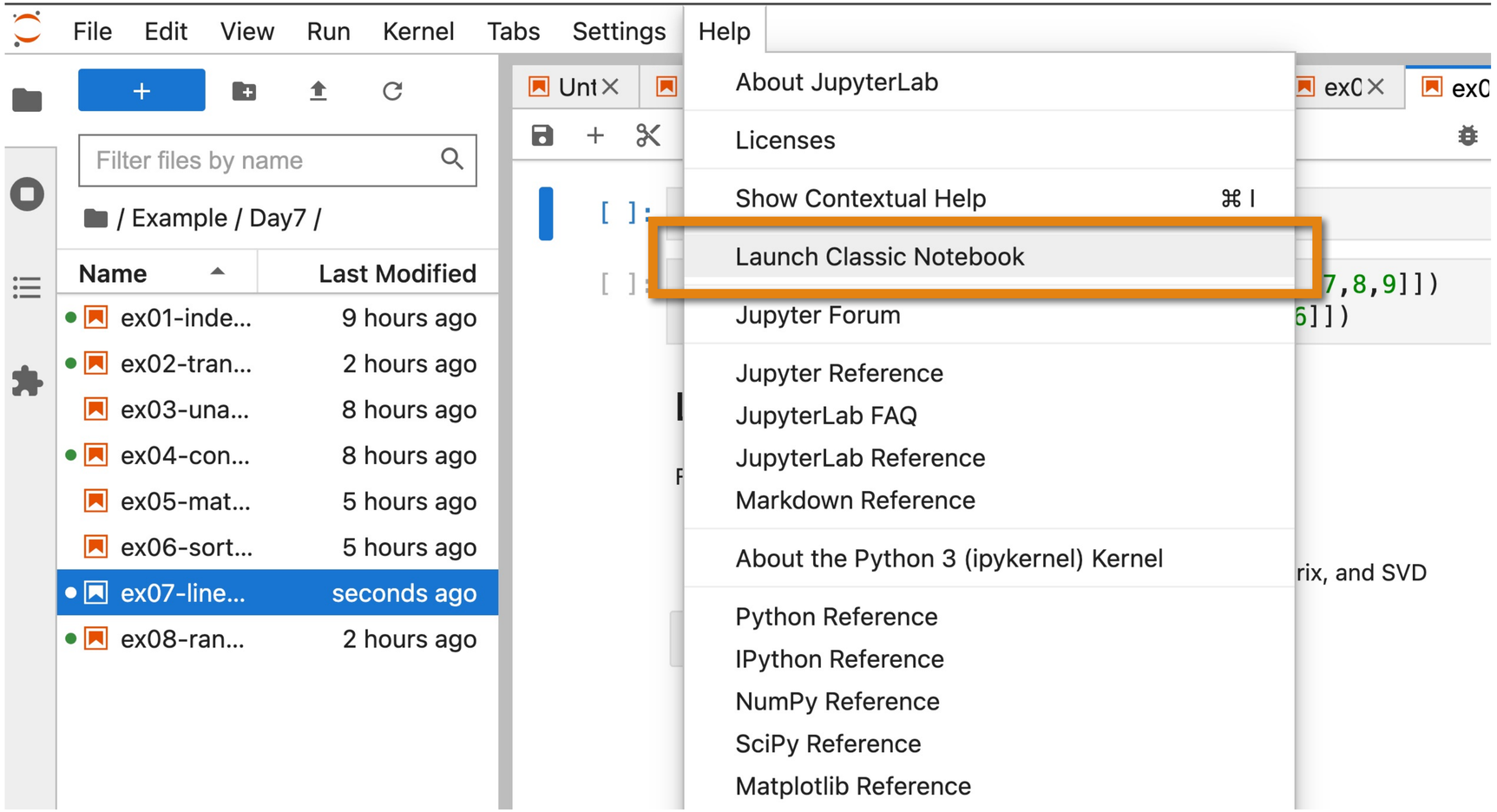
3. **Enable nbextension on Jupyter**

   ```
   jupyter nbextensions_configurator enable
   ```

4. **Install autopep8 which comes with pycodestyle**
   conda install -c conda-forge autopep8

https://codeburst.io/jupyter-notebook-tricks-for-data-science-that-enhance-your-efficiency-95f98d3adee4
https://jupyter-contrib-nbextensions.readthedocs.io/en/latest/install.html

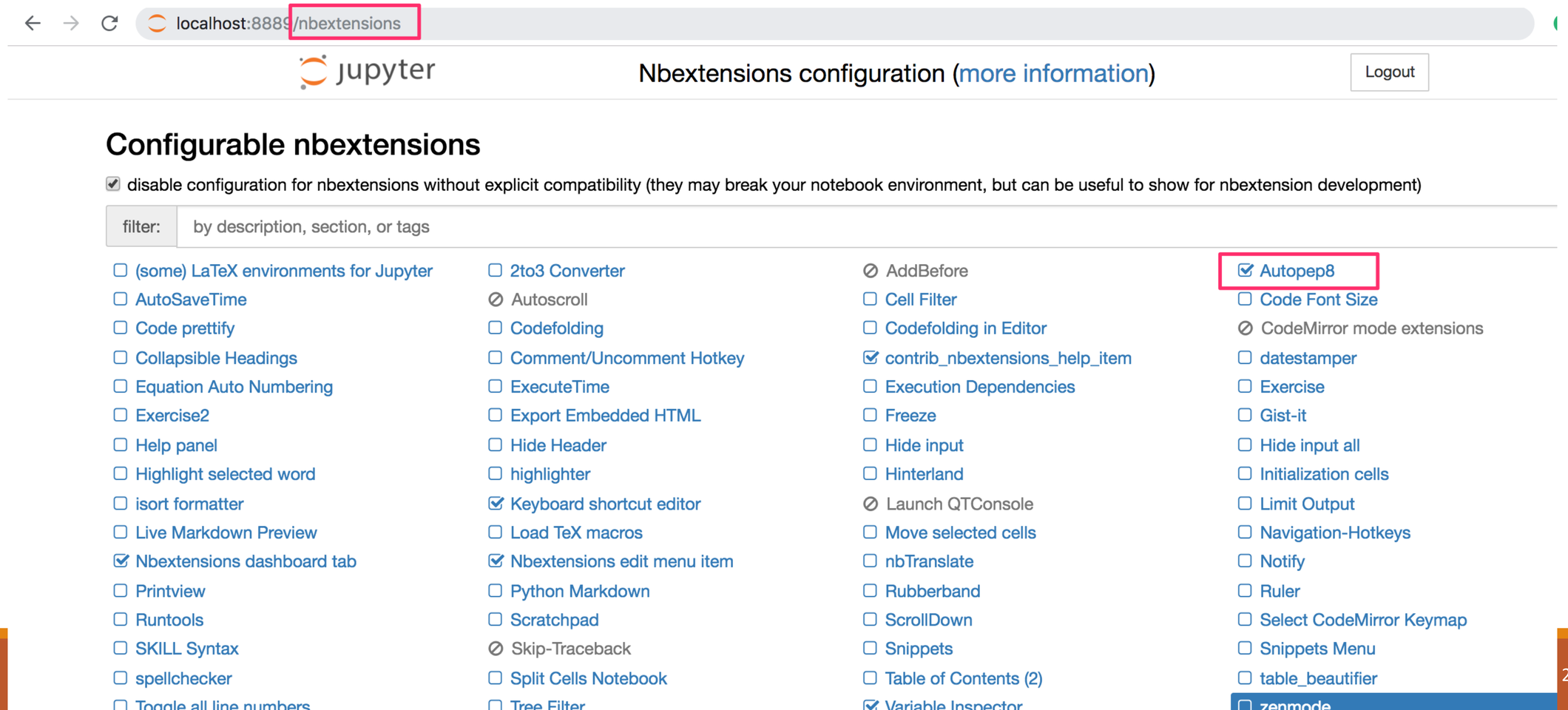
Codestyle Check (autopep8) on Jupyter Notebook



https://codeburst.io/jupyter-notebook-tricks-for-data-science-that-enhance-your-efficiency-95f98d3adee4
https://jupyter-contrib-nbextensions.readthedocs.io/en/latest/install.html

Codestyle Check (autopep8) on Jupyter Notebook

Enable Autopep8



Codestyle Check (autopep8) on Jupyter Notebook

To update the code with PEP8, choose a cell and click 

